



25. Quadratic Discriminant Analysis

When there is non-linear, LDA fails, so we need a better one

Previous Section:

[24. Linear Discriminant Analysis](#)

Contents:

[1. Quadratic Discriminant Analysis – The Overview](#)

[2. Quadratic Discriminant Analysis – Same Dataset, Different Approach](#)

[3. Quadratic Discriminant Analysis – Discriminant Function](#)

[4. Quadratic Discriminant Analysis with Python](#)

[Summary](#)

[References](#)

Let's assume that you've made it through Linear Discriminant Analysis (*If not, please go read it*). And if you're being honest with yourself, something probably felt a bit off. LDA is elegant. Clean. Almost *too* clean. It draws a straight line, and hopes the world agrees. But reality doesn't always care about your straight lines. Sometimes, the data bends. Sometimes, it curves. Sometimes, the boundary you *need*, simply isn't linear.

That's where things start to break. LDA makes a very strong assumption: All classes share the same covariance (same "shape", same spread). In other words, it forces everything into a single geometric structure, even when the data clearly disagrees. And when that assumption fails? The boundary fails with it.

So instead of forcing the world to stay linear, we take a different approach. We let each class speak for itself. We stop pretending they all behave the same. We stop compressing everything into one shared structure.

We embrace the complexity. That's where **Quadratic Discriminant Analysis (QDA)** comes in. Unlike LDA, QDA doesn't simplify the problem. It actually makes it harder.

- Each class gets its own covariance matrix.
- Each class gets its own shape.
- Each class defines its own distribution.

No shortcuts. No shared assumptions. And because of that... the decision boundary is no longer a straight line. It curves. It adapts. It fits the data the way the data actually *is*... not the way we wish it was.

But of course, nothing comes for free. More flexibility means more parameters. More parameters means more data required. And more data means more risk if you don't have enough of it.

So QDA is not "better" than LDA. It's just more honest. It admits that the world is messy. And it tries to model that mess directly.

1. Quadratic Discriminant Analysis – The Overview

Quadratic Discriminant Analysis (QDA) can be seen as the more flexible—and more demanding—extension of Linear Discriminant Analysis (LDA).

Where LDA makes a simplifying assumption that all classes share the same covariance matrix (in other words, they have the same “shape” in space), QDA removes that restriction entirely. Instead, each class is allowed to have its own covariance matrix.

This might sound like a small change, but it completely transforms the model. Because once each class has its own shape and spread, the decision boundary is no longer forced to be a straight line. It becomes **quadratic**. It curves. It adapts to the structure of each class individually.

And this is exactly why QDA performs better in situations where the data is not linearly separable. It doesn't try to “flatten” the problem into a linear boundary. It follows the natural geometry of the data.

But as always, flexibility comes with a cost. Allowing each class to have its own covariance matrix means the model must estimate many more parameters. And that leads to two important consequences:

- QDA requires more data to make reliable estimates
- QDA is more prone to overfitting, especially on small datasets

So while QDA can capture more complex patterns than LDA, it can also fail more easily if the data is limited.

A simple example (Student classification) - Imagine you are trying to classify students into two groups based on their study behavior:

- Class A → Students who study consistently
- Class B → Students who cram before exams

Now suppose you use two features:

- Hours studied per week
- Sleep hours before exams

In reality, these two groups behave very differently:

- Consistent students might have a stable pattern → moderate study hours and stable sleep
- Cramming students might show extreme variation → low study hours normally, then spikes before exams, and irregular sleep

This means:

- The “shape” of Class A is compact and stable
- The “shape” of Class B is more spread out and irregular

If you apply LDA, it will force both groups to share the same covariance structure. As a result, we obtain a straight-line boundary that may not separate them well

But QDA allows Class A to have a tight covariance, and Class B to have a wider, more scattered covariance. And because of that, the boundary curves around the data. And it adapts to the true behavior of each group

Which leads to better classification—if you have enough data to support it

So in the end, QDA is not just “LDA but better.” → It's LDA without the simplifying assumptions. More expressive. More flexible. But also more fragile.

2. Quadratic Discriminant Analysis – Same Dataset, Different Approach

To understand how QDA actually works, we'll reuse the same small dataset from LDA. Nothing changes at the beginning. But very quickly, you'll see where QDA takes a different path.

StudentID	School Year	English Level	Part-Time Job
001	1	5	No
002	2	1	Yes
003	2	2	Yes
004	2	3	Yes

StudentID	School Year	English Level	Part-Time Job
005	1	3	No
006	1	3	No
007	2	3	Yes
008	1	2	No
009	1	4	No
010	1	1	Yes

Step 1: Compute Class-wise Mean Vectors

Just like in LDA, we begin by summarizing each class using its mean vector.

• Class: Yes (Has Part-Time Job)

School Year	English Level
2	1
2	2
2	3
1	1

$$\mu_1 = \begin{bmatrix} \frac{2+2+2+1}{4} \\ \frac{1+2+3+1}{4} \end{bmatrix} = \begin{bmatrix} 1.75 \\ 1.75 \end{bmatrix}$$

• Class: No (No Part-Time Job)

School Year	English Level
1	5
2	3
1	3
1	2
1	4
1	1

$$\mu_2 = \begin{bmatrix} \frac{1+2+1+1+1+1}{6} \\ \frac{5+3+3+2+4+1}{6} \end{bmatrix} = \begin{bmatrix} 1.17 \\ 3.00 \end{bmatrix}$$

Step 2: Mean Difference (for intuition)

Before moving forward, it's helpful to observe how far apart the two classes are:

$$(\mu_1 - \mu_2)^2 = \begin{bmatrix} (1.75 - 1.17)^2 \\ (1.75 - 3.00)^2 \end{bmatrix} = \begin{bmatrix} 0.3364 \\ 1.5625 \end{bmatrix}$$

This gives us a rough idea of which feature separates the classes more.

Step 3: Compute Class-Specific Covariance Matrices

Here is where QDA diverges from LDA. Instead of sharing one covariance matrix, each class gets its **own** covariance matrix.

• Covariance Matrix for Class: Yes

Mean vector: $\mu_1 = (1.75, 1.75)$

(School, English)	Deviation ($x_i - \mu_1$)
(2, 1)	(0.25, -0.75)

(School, English)	Deviation ($x_i - \mu_1$)
(2, 2)	(0.25, 0.25)
(2, 3)	(0.25, 1.25)
(1, 1)	(-0.75, -0.75)

Covariance formula:

$$\Sigma_k = \frac{1}{n_k} \sum_{i=1}^{n_k} (x_i - \mu_k)(x_i - \mu_k)^T$$

Result: $\Sigma_1 = \begin{bmatrix} 0.1875 & 0.1875 \\ 0.1875 & 0.6875 \end{bmatrix}$

• Covariance Matrix for Class: No

Mean vector: $\mu_2 = (1.17, 3.00)$

(School, English)	Deviation ($x_i - \mu_2$)
(1, 5)	(-0.17, 2.00)
(2, 3)	(0.83, 0.00)
(1, 3)	(-0.17, 0.00)
(1, 2)	(-0.17, -1.00)
(1, 4)	(-0.17, 1.00)
(1, 1)	(-0.17, -2.00)

Result: $\Sigma_2 = \begin{bmatrix} 0.1379 & 0 \\ 0 & 1.667 \end{bmatrix}$

Step 4: Prior Probabilities

$$\pi_{\text{Yes}} = \frac{4}{10} = 0.4 \quad ; \quad \pi_{\text{No}} = \frac{6}{10} = 0.6$$

And now, everything is ready. We have mean vectors, class-specific covariance matrices, prior probabilities. Which leads us to the core of QDA.

3. Quadratic Discriminant Analysis – Discriminant Function

This is where everything comes together. All the means, all the covariance matrices. They weren't just for show. They feed directly into the **QDA discriminant function**—the rule that decides which class a new data point belongs to.

General QDA Formula

$$\delta_k(x) = -\frac{1}{2} \log |\Sigma_k| - \frac{1}{2} (x - \mu_k)^T \Sigma_k^{-1} (x - \mu_k) + \log(\pi_k)$$

Where: $x = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} \text{School Year} \\ \text{English Level} \end{bmatrix}$

- $|\Sigma_k|$: Determinant of covariance matrix
- Σ_k^{-1} : Inverse covariance matrix
- μ_k : Mean vector of class k
- π_k : Prior probability

Class: Yes - Given: $\pi_1 = 0.4$; $\mu_1 = \begin{bmatrix} 1.75 \\ 1.75 \end{bmatrix}$

Covariance Matrix Σ_1	Determinant	Inverse
$\begin{bmatrix} 0.1875 & 0.1875 \\ 0.1875 & 0.6875 \end{bmatrix}$	0.09375	$\begin{bmatrix} 7.333 & -2 \\ -2 & 2 \end{bmatrix}$

Substitute into the QDA formula:

$$\delta_{yes}(x) = -\frac{1}{2} \log(0.09375) - \frac{1}{2} (x - \mu_1)^T \Sigma_1^{-1} (x - \mu_1) + \log(0.4)$$

Expanding the Quadratic Form: $(x - \mu_1)^T \Sigma_1^{-1} (x - \mu_1)$

$$\begin{aligned} &= \begin{bmatrix} x_1 - 1.75 & x_2 - 1.75 \end{bmatrix} \begin{bmatrix} 7.333 & -2 \\ -2 & 2 \end{bmatrix} \begin{bmatrix} x_1 - 1.75 & x_2 - 1.75 \end{bmatrix} \\ &= (x_1 - 1.75)[7.333(x_1 - 1.75) - 2(x_2 - 1.75)] + (x_2 - 1.75)[-2(x_1 - 1.75) + 2(x_2 - 1.75)] \\ &= 7.333(x_1 - 1.75)^2 - 4(x_1 - 1.75)(x_2 - 1.75) + 2(x_2 - 1.75)^2 \\ &\rightarrow \text{Notice something important: There is a **cross-term** } (x_1 - 1.75)(x_2 - 1.75) \end{aligned}$$

This is the moment QDA reveals itself. This is what creates a **curved (quadratic) boundary**

Class: No - Given: $\pi_2 = 0.6$; $\mu_2 = \begin{bmatrix} 1.17 \\ 3.00 \end{bmatrix}$

Covariance Matrix Σ_2	Determinant	Inverse
$\begin{bmatrix} 0.1379 & 0 \\ 0 & 1.667 \end{bmatrix}$	0.2315	$\begin{bmatrix} 7.199 & 0 \\ 0 & 0.6 \end{bmatrix}$

Substitute into the QDA formula:

$$\delta_{No}(x) = -\frac{1}{2} \log(0.2315) - \frac{1}{2} (x - \mu_2)^T \Sigma_2^{-1} (x - \mu_2) + \log(0.6)$$

Expanding the Quadratic Form: $(x - \mu_2)^T \Sigma_2^{-1} (x - \mu_2)$

$$\begin{aligned} &= (x - \mu_2)^T \Sigma_2^{-1} (x - \mu_2) = \begin{bmatrix} x_1 - 1.17 & x_2 - 3.00 \end{bmatrix} \begin{bmatrix} 7.199 & 0 \\ 0 & 0.6 \end{bmatrix} \begin{bmatrix} x_1 - 1.17 \\ x_2 - 3.00 \end{bmatrix} \\ &= 7.199(x_1 - 1.17)^2 + 0.6(x_2 - 3)^2 \end{aligned}$$

Here's the contrast: **No cross-term**. Because covariance between *features* = 0. So this class produces a simpler quadratic shape.

Final Discriminant Functions

- **Class: Yes**

$$\delta_{Yes}(x) = -\frac{1}{2} \log(0.09375) - \frac{1}{2} \left[7.333(x_1 - 1.75)^2 - 4(x_1 - 1.75)(x_2 - 1.75) + 2(x_2 - 1.75)^2 \right] + \log(0.4)$$

- **Class: No**

$$\delta_{No}(x) = -\frac{1}{2} \log(0.2315) - \frac{1}{2} \left[7.199(x_1 - 1.17)^2 + 0.6(x_2 - 3)^2 \right] + \log(0.6)$$

Step 4: Making Predictions

Now comes the final step. We simply compute both scores: $\delta_{Yes}(x)$ and $\delta_{No}(x)$ → And choose the **larger one**.

Test Samples

StudentID	School Year (x_1)	English Level (x_2)
032	1	5
033	2	1
034	2	2

Predictions

- **Sample 032**

- $\delta_{\text{Yes}}(x) = -5.2326$
- $\delta_{\text{No}}(x) = 0.1167$

⇒ **Prediction: NO**

- **Sample 033**

- $\delta_{\text{Yes}}(x) = -0.8993$
- $\delta_{\text{No}}(x) = -3.4589$

⇒ **Prediction: YES**

- **Sample 034**

- $\delta_{\text{Yes}}(x) = -1.2326$
- $\delta_{\text{No}}(x) = -1.08326$

⇒ **Prediction: NO**

This completes a full, from-scratch implementation of QDA. And if you compare this with LDA, one difference stands out immediately: LDA gives you a straight line. QDA gives you a curve. And that curve exists for one reason only:

Each class is allowed to have its own covariance structure.

Which means QDA doesn't force the data into a shape. It follows it.

4. Quadratic Discriminant Analysis with Python

Linear Discriminant Analysis (or LDA) is straightforward using the Scikit-Learn. Using a larger student dataset than the one we use for LDA in the previous section:

student.csv

```
import pandas as pd

df = pd.read_csv('student.csv').set_index('StudentID')
# Replace the Yes No in Part Time Job
df['Part_Time_Job'] = df['Part_Time_Job'].map({'Yes': 1, 'No': 0})
# One-Hot Encoding on Major
df = pd.get_dummies(df, columns=['Major']).replace({True: 1, False: 0})
```

```
from sklearn.model_selection import train_test_split
```

```
# Features and Target
target = 'Part_Time_Job'
features = df.columns.drop(target)
# 80% training and 20% test
train_df, test_df = train_test_split(df, test_size=0.2, random_state=42)
X_train = train_df[features]
X_test = test_df[features]
y_train = train_df[target]
y_test = test_df[target]
```

```
# Train a Quadratic Discriminant Analysis model with all features
from sklearn.discriminant_analysis import QuadraticDiscriminantAnalysis
from sklearn.metrics import accuracy_score

X_train = train_df[features]
X_test = test_df[features]
qda = QuadraticDiscriminantAnalysis()
qda.fit(X_train, y_train)
y_pred_qda = qda.predict(X_test)

accuracy_qda = accuracy_score(y_test, y_pred_qda)
print("Accuracy:", accuracy_qda)
```

Accuracy: 0.95

```
# Optionally, you can also train Linear Discriminant Analysis model with all features
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis

lda = LinearDiscriminantAnalysis()
lda.fit(X_train, y_train)
y_pred_lda = lda.predict(X_test)

accuracy_lda = accuracy_score(y_test, y_pred_lda)
print("Accuracy:", accuracy_lda)
```

Accuracy: 0.75

```
# Plot the Decision Boundary for only two features
import numpy as np
import matplotlib.pyplot as plt
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis, QuadraticDiscriminantAnalysis

# Select two features
plot_features = ['Gpa_Score', 'Fitness_Score']

X = df[plot_features]
y = df['Part_Time_Job']

# Retrain models with only the two selected features for plotting
lda_plot = LinearDiscriminantAnalysis()
qda_plot = QuadraticDiscriminantAnalysis()
```

```

lda_plot.fit(X, y)
qda_plot.fit(X, y)

# Create mesh grid
x_min, x_max = X.iloc[:, 0].min() - 0.5, X.iloc[:, 0].max() + 0.5
y_min, y_max = X.iloc[:, 1].min() - 0.5, X.iloc[:, 1].max() + 0.5

xx, yy = np.meshgrid(np.linspace(x_min, x_max, 300),
                     np.linspace(y_min, y_max, 300))

# Predict for both models
Z_lda = lda_plot.predict(np.c_[xx.ravel(), yy.ravel()]).reshape(xx.shape)
Z_qda = qda_plot.predict(np.c_[xx.ravel(), yy.ravel()]).reshape(xx.shape)

# Create subplots
fig, axes = plt.subplots(1, 2, figsize=(14, 6))

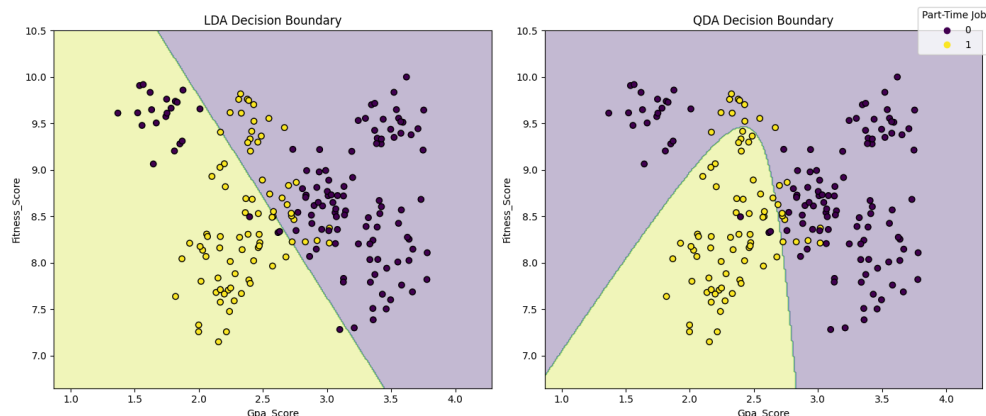
# LDA Plot
axes[0].contourf(xx, yy, Z_lda, alpha=0.3)
scatter1 = axes[0].scatter(X.iloc[:, 0], X.iloc[:, 1], c=y, edgecolor='k')
axes[0].set_title('LDA Decision Boundary')
axes[0].set_xlabel('Gpa_Score')
axes[0].set_ylabel('Fitness_Score')

# QDA Plot
axes[1].contourf(xx, yy, Z_qda, alpha=0.3)
scatter2 = axes[1].scatter(X.iloc[:, 0], X.iloc[:, 1], c=y, edgecolor='k')
axes[1].set_title('QDA Decision Boundary')
axes[1].set_xlabel('Gpa_Score')
axes[1].set_ylabel('Fitness_Score')

# Shared legend
handles, labels = scatter1.legend_elements()
fig.legend(handles, labels, title="Part-Time Job", loc="upper right")

plt.tight_layout()
plt.show()

```



Summary

QDA extends from LDA by assuming each class has its own covariance. But that small change? It changes everything. Instead of forcing all classes to share the same shape, QDA lets each class define its own structure.

And because of that: The decision boundary is no longer linear. It becomes quadratic, curved, and more adaptive

This makes QDA far more powerful when the data is not linearly separable. It doesn't try to simplify reality. It follows it.

But there's always a trade-off. More flexibility means more parameters to estimate, more data required, more risk of overfitting

So QDA is not a "better LDA." It's a more expressive model that works beautifully when you have enough data, and breaks just as easily when you don't.

References

<https://www.geeksforgeeks.org/data-analysis/quadratic-discriminant-analysis/>

<https://www.kaggle.com/discussions/general/448328>

Next Section:

 [26. Principal Component Analysis](#)